MPS-P Series Programmable DC Power Supply Modbus Communication Protocol

The register address and corresponding function code table are as follows：

Register Address	Name	Function	Function Code 0x03	Function Code 0X06	Function Code 0X10	Value Range
0X0000	Remote Mode	Remote Operation Mode	Y	Y	Y	0: Local Mode 1: Remote Mode
0X0001	V Min	Minimum Voltage Setting	Y	Y	Y	0~Vmax Step: 1mV
0X0002	V Max	Maximum Voltage Setting	Y	Y	Y	0~Vmax Step: 1mV
0X0003	A Min	Minimum Current Setting	Y	Y	Y	0~Amax Step: 1mA
0X0004	A Max	Maximum Current Setting	Y	Y	Y	0~Amax Step: 1mA
0X0005	V Set	Voltage Setting	Y	Y	Y	0~Vmax Step: 1mV
0X0006	A Set	Current Setting	Y	Y	Y	0~Amax Step: 1mA
0X0007	OVP Set	Over-Voltage Setting	Y	Y	Y	0~Vmax+1V Step: 1mV
0X0008	OCP Set	Over-Current Setting	Y	Y	Y	0~Amax+1A Step: 1mA
0X0009	OVP State	Over-Voltage Protection Status	Y	Y	Y	0: Disable 1: Enable
0X000A	OCP State	Over-Current Protection Status	Y	Y	Y	0: Disable 1: Enable
0X000B	Output State	Output Status	Y	Y	Y	0: Disable 1: Enable
0X000C	Buzzer State	Buzzer Status	Y	Y	Y	0: Disable 1: Enable
0X000D	Sense State	Remote Compensation Status	Y	Y	Y	0: Disable 1: Enable
0X000E	Vout	Actual Output Voltage	Y	N	N	0~Vmax Step: 1mV
0X000F	Aout	Actual Output Current	Y	N	N	0~Amax Step: 1mA
0X0010	CV/CC	CV/CC Status	Y	N	N	0: CV Mode 1:CC Mode

Note: All operations are effective only when in remote operation mode (the value of register 0X0000 is 0X0001).

After the power supply is restarted, it defaults to local operation mode.

EmailSupport: service@szmatrix.com

The function code table and interval time are as follows:

Function Code	Corresponding Function	Interval Between Operations
0X03	Read Multiple Registers Command	N * 5ms
0X06	Write Single Register Command	(Registers 0x0000~0x0021) 10ms, (Registers 0x0065~0x02BC) 300ms
0X10	Write Multiple Registers Command	N * 5ms

Note: N is the number of registers.

The communication protocol format is as follows:

Function Code 0x03

PC Sends: 8 Byte

Address	Function Code	Starting Address High Byte	Starting Address Low Byte	Number of Registers High Byte	Number of Registers Low Byte	CRC16 Checksum Low Byte	CRC16 Checksum High Byte
0X03	0X03						

Power Supply Returns: 5 + N*2 Bytes

Address	Function Code	Data Byte Count	Return Data High Byte	Return Data Low Byte	Return Data N+1 High Byte	Return Data N+1 Low Byte	CRC16 Checksum Low Byte	CRC16 Checksum High Byte
0X03	0X03							

*Address range: (0X0000 to 0X0010)

Example 1: Read Single Register Status (0X0000)

PC Sends: 03 03 00 00 00 01 85 E8 (Starting address is 0X0000, number of registers is 0X0001, CRC16 checksum result is 0XE885)

Power Supply Returns: 03 03 02 00 01 00 44 (02 indicates the data length is 2 bytes, 0X0001 indicates remote operation mode, 0X4400 is the CRC16 checksum result)

Function Code 0X06

PC Sends: 8 Byte

Address	Function Code	Write Address High Byte	Write Address Low Byte	Write Data High Byte	Write Data Low Byte	CRC16 Checksum Low Byte	CRC16 Checksum High Byte
0X03	0X06						

Power Supply Returns: 8Byte

Address	Function Code	Write Address High Byte	Write Address Low Byte	Write Data High Byte	Write Data Low Byte	CRC16 Checksum Low Byte	CRC16 Checksum High Byte
0X03	0X06						

*Address range: (0X0000~0X000D)

Example 1: Write Single Register Status (0X0000)

PC Sends: 03 06 00 00 00 01 49 E8 (Write address is 0X0000, write data is 0X0001, CRC16 checksum result is 0XE849)

Power Supply Returns: 03 06 00 00 00 01 49 E8 (Write address is 0X0000, write data is 0X0001, CRC16 checksum result is 0XE849)

Function Code 0X10

PC Sends: 9+N*2 Byte

Address	Function Code	Starting Address High Byte	Starting Address Low Byte	Number of Registers High Byte	Number of Registers Low Byte	Data Byte Count	Write Data High Byte	Write Data Low Byte	Write Data N+1 High Byte	Write Data N+1 Low Byte	CRC16 Checksum Low Byte	CRC16 Checksum High Byte
0X03	0X10											

Power Supply Returns: 8 Byte

Address	Function Code	Starting Address High Byte	Starting Address Low Byte	Number of Registers High Byte	Number of Registers Low Byte	CRC16 Checksum Low Byte	CRC16 Checksum High Byte
0X03	0X10						

*Address range: (0X0000~0X000D)

Example 1: Write Single Register Status(0X0001)
PC Sends: 03 10 00 01 00 01 02 03 E8 BE 5F (Starting address is 0X0001, number of registers is 0X0001,write data is 0X03E8,CRC16 checksum result is 0X5FBE)
Power Supply Returns: 03 10 00 01 00 01 51 EB (Starting address is 0X0001, number of registers is 0X0001, CRC16 checksum result is 0XEB51)

举例二：写多个寄存器状态(0X0001~0X0003)
PC 发送：03 10 00 01 00 03 03 E8 07 D0 0B B8 6D BD (Starting address is 0X0001, number of registers is 0X0003,CRC16 checksum result is 0XBD6D)
电源返回：03 10 00 01 00 03 D0 2A (Starting address is 0X0001, number of registers is 0X0003, CRC16 checksum result is 0X2AD0)

CRC16 Calculation Code:

```
const unsigned char  CRCHTalbe[] =
{
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
```

```
0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x00, 0xC1, 0x81, 0x40,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40, 0x01, 0xC0, 0x80, 0x41, 0x01, 0xC0, 0x80, 0x41,
0x00, 0xC1, 0x81, 0x40
}


const unsigned char  CRCLTalbe[] =
{
0x00, 0xC0, 0xC1, 0x01, 0xC3, 0x03, 0x02, 0xC2, 0xC6, 0x06, 0x07, 0xC7,
0x05, 0xC5, 0xC4, 0x04, 0xCC, 0x0C, 0x0D, 0xCD, 0x0F, 0xCF, 0xCE, 0x0E,
0x0A, 0xCA, 0xCB, 0x0B, 0xC9, 0x09, 0x08, 0xC8, 0xD8, 0x18, 0x19, 0xD9,
0x1B, 0xDB, 0xDA, 0x1A, 0x1E, 0xDE, 0xDF, 0x1F, 0xDD, 0x1D, 0x1C, 0xDC,
0x14, 0xD4, 0xD5, 0x15, 0xD7, 0x17, 0x16, 0xD6, 0xD2, 0x12, 0x13, 0xD3,
0x11, 0xD1, 0xD0, 0x10, 0xF0, 0x30, 0x31, 0xF1, 0x33, 0xF3, 0xF2, 0x32,
0x36, 0xF6, 0xF7, 0x37, 0xF5, 0x35, 0x34, 0xF4, 0x3C, 0xFC, 0xFD, 0x3D,
0xFF, 0x3F, 0x3E, 0xFE, 0xFA, 0x3A, 0x3B, 0xFB, 0x39, 0xF9, 0xF8, 0x38,
0x28, 0xE8, 0xE9, 0x29, 0xEB, 0x2B, 0x2A, 0xEA, 0xEE, 0x2E, 0x2F, 0xEF,
0x2D, 0xED, 0xEC, 0x2C, 0xE4, 0x24, 0x25, 0xE5, 0x27, 0xE7, 0xE6, 0x26,
0x22, 0xE2, 0xE3, 0x23, 0xE1, 0x21, 0x20, 0xE0, 0xA0, 0x60, 0x61, 0xA1,
0x63, 0xA3, 0xA2, 0x62, 0x66, 0xA6, 0xA7, 0x67, 0xA5, 0x65, 0x64, 0xA4,
0x6C, 0xAC, 0xAD, 0x6D, 0xAF, 0x6F, 0x6E, 0xAE, 0xAA, 0x6A, 0x6B, 0xAB,
0x69, 0xA9, 0xA8, 0x68, 0x78, 0xB8, 0xB9, 0x79, 0xBB, 0x7B, 0x7A, 0xBA,
```

```
0xBE, 0x7E, 0x7F, 0xBF, 0x7D, 0xBD, 0xBC, 0x7C, 0xB4, 0x74, 0x75, 0xB5,
0x77, 0xB7, 0xB6, 0x76, 0x72, 0xB2, 0xB3, 0x73, 0xB1, 0x71, 0x70, 0xB0,
0x50, 0x90, 0x91, 0x51, 0x93, 0x53, 0x52, 0x92, 0x96, 0x56, 0x57, 0x97,
0x55, 0x95, 0x94, 0x54, 0x9C, 0x5C, 0x5D, 0x9D, 0x5F, 0x9F, 0x9E, 0x5E,
0x5A, 0x9A, 0x9B, 0x5B, 0x99, 0x59, 0x58, 0x98, 0x88, 0x48, 0x49, 0x89,
0x4B, 0x8B, 0x8A, 0x4A, 0x4E, 0x8E, 0x8F, 0x4F, 0x8D, 0x4D, 0x4C, 0x8C,
0x44, 0x84, 0x85, 0x45, 0x87, 0x47, 0x46, 0x86, 0x82, 0x42, 0x43, 0x83,
0x41, 0x81, 0x80, 0x40
};
```

```
unsigned int crc16(unsigned char *DData,unsigned char len)
{
   unsigned char CRCHi = 0XFF;
   unsigned char CRCLo = 0XFF;
   unsigned int wIndex;
   unsigned int CRC_DData;
   while(len--)
    {
    wIndex = CRCLo ^ *DData++;
    CRCLo = CRCHi ^ CRCHTalbe[wIndex];
    CRCHi = CRCLTalbe[wIndex];
    }
CRC_DData = CRCHi;
CRC_DData<<= 8;
CRC_DData |=CRCLo;
   return CRC_DData;
}
```